

Proyecto Técnico: Sistema Integral de Gestión "Micro-Market"

1. Objetivo del Proyecto

Desarrollar una API REST robusta utilizando **Spring Boot** para la gestión administrativa y operativa de un supermercado. El sistema debe permitir el control de inventarios, la gestión de proveedores, el registro de personal y el procesamiento de ventas.

2. Requerimientos Técnicos

- **Lenguaje y Framework:** Java 17+ con Spring Boot 3.x.
 - **Gestión de Dependencias:** Maven.
 - **Persistencia:** Spring Data JPA con motor de base de datos relacional (MySQL).
 - **Arquitectura de Software:**
 - Uso estricto de **DTOs** (Data Transfer Objects) para el intercambio de datos en los controladores.
 - Separación de responsabilidades en capas: `Controller`, `Service`, `Repository` y `Entity`.
 - Manejo Global de Excepciones.
 - **Validaciones:** Uso de anotaciones `@Valid` y `javax.validation.constraints` en todos los modelos de entrada.
-

3. Módulos y Requerimientos Funcionales

Módulo I: Inventario y Productos

- **Entidades:** Productos y Categorías.
- **Acciones:** CRUD completo para ambas entidades.

- **Regla de Negocio 1:** No se permite la eliminación física de registros de productos. Se debe implementar **Borrado Lógico** (Soft Delete) mediante un atributo booleano de estado.
- **Regla de Negocio 2:** Los productos deben tener un código de barras único y validado. No se pueden registrar duplicados.
- **Regla de Negocio 3:** Al consultar una categoría, la respuesta debe incluir la lista de todos los productos vinculados a ella.

Módulo II: Proveedores y Abastecimiento

- **Entidades:** Proveedores.
- **Relación:** Implementar una relación **ManyToMany** entre Productos y Proveedores.
- **Regla de Negocio 1:** Endpoint específico para "Entrada de Almacén" que reciba el ID del producto, el ID del proveedor y la cantidad. Esta acción debe sumar las unidades al stock actual.
- **Regla de Negocio 2:** El NIT del proveedor es un campo obligatorio y no puede repetirse en la base de datos.

Módulo III: Gestión de Personal (Empleados)

- **Entidad:** Empleados (Cédula, Nombre, Cargo, Fecha de Ingreso, Salario).
- **Regla de Negocio 1:** El sistema solo debe permitir los cargos: **ADMINISTRADOR**, **CAJERO** y **AUXILIAR**.
- **Regla de Negocio 2:** Endpoint de consulta para listar empleados según su cargo o por rango de fecha de ingreso.

Módulo IV: Facturación y Ventas

- **Entidades:** Venta (Cabecera) y Detalle de Venta.
- **Relaciones:** La Venta debe estar vinculada obligatoriamente a un **Empleado** (quien realiza la transacción).
- **Regla de Negocio 1:** Al procesar una venta, el sistema debe **restar automáticamente** la cantidad vendida del stock disponible en el Módulo I.
- **Regla de Negocio 2:** Si un producto no tiene stock suficiente para cubrir la venta, el sistema debe retornar un error **400 Bad Request** y cancelar la transacción.

- **Regla de Negocio 3:** La factura debe calcular automáticamente el total, subtotal e IVA (19%).
-

4. Reglas de Git y Flujo de Trabajo

- **Estándar de Mensajería:** Uso obligatorio de **Conventional Commits**:
 - `feat:` para nuevas funcionalidades.
 - `fix:` para corrección de errores.
 - `docs:` para cambios en documentación.
 - `refactor:` para mejoras de código que no afectan la funcionalidad.
 - **Estructura de Ramas:**
 - `main` : Rama protegida para el código final estable.
 - `develop` : Rama de integración grupal.
 - `feature/nombre-funcionalidad` : Ramas temporales para el desarrollo de cada módulo.
 - **Integración:** Queda prohibido el *merge* directo a `main` . Todo cambio debe pasar por un **Pull Request (PR)** que debe ser revisado y aprobado por al menos un integrante del equipo distinto al autor.
-

5. Restricciones del Proyecto

- **Datos:** Se debe incluir un archivo `data.sql` y una colección de **Postman** que contenga datos de prueba para todos los módulos.